

Problem Set 5

● Graded

12 Hours, 57 Minutes Late

Student

Jinku Jung

Total Points

28 / 30 pts

Question 1

7.11

5 / 5 pts

✓ + 2 pts a) Correct

✓ + 2 pts b) Correct

✓ + 1 pt Showed work

🗨 +5/5 Great work!

Question 2

7.12

5 / 5 pts

✓ + 1 pt Presents a clear answer

✓ + 2 pts Correct first advantage

✓ + 2 pts Correct second advantage

🗨 Awesome job!

Question 3

7.16

3 / 5 pts

✓ + 1 pt Correct number of paths

🗨 + 2 pts Great job!
b), c) +1/2 each - need to show work with Dijkstra's algorithm

Question 4

7.17

5 / 5 pts

✓ + 1 pt Presents a clear and specific answer

✓ + 2 pts States that the heuristic will not always deliver the message

✓ + 2 pts Correct explanation/counterexample shown

Question 5

7.20

5 / 5 pts

✓ + 1 pt Presents a clear and specific answer

✓ + 2 pts Some correct first advantage

✓ + 2 pts Some correct second advantage

Question 6

7.21

5 / 5 pts

🗨 + 5 pts Point adjustment

Question 7

Format and tagging

0 / 0 pts

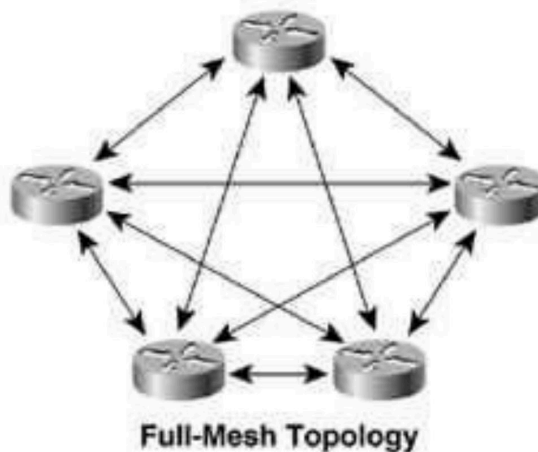
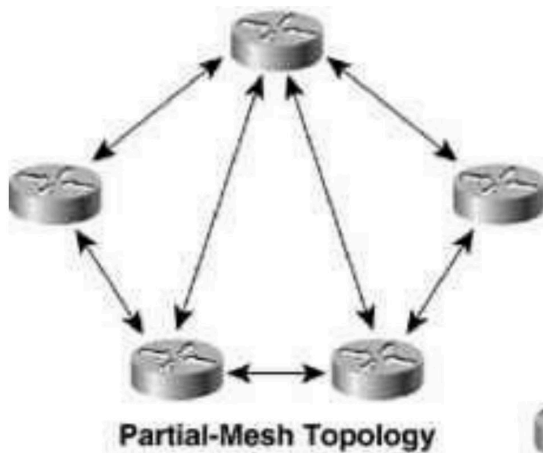
✓ + 0 pts Correct

Question assigned to the following page: [1](#)

Problem Set #5

Ch. 7/#11:

- a.) That would mean using a network topology with zero redundant communication links. So, if you use a bus, star, or ring topology, given 5 WAN nodes:
- i.) You would need 4 unique communication links between them.
 - ii.) Thus, if you extrapolate this result, the general case answer should be “N-1” communications links required for the least number of connection links used, per “N” number of given WAN nodes (ex: routers), where $N \geq 2$.
- b.) If a particular connection link were to go “offline” one possible solution would be to use another redundant link path, in tandem, with the original connection link along the very same two nodes (via EtherChannel bonding on a Cisco IOS interface). Ideally though, over a medium LAN to large WAN network, a **partial mesh** or **full mesh** network topology (see graphic below) would be the best choice (if monetary cost was NOT an issue and network redundancy was MISSION CRITICAL):



Question assigned to the following page: [2](#)

Problem Set #5

Ch. 7/#12:

The primary advantage of network redundancy becomes apparent when:

- 1.) When a connection failure occurs:
 - a.) If there exist at least two unique paths from a given node A to node B, then one link-failure of a tandem connection link via EtherChannel bonding will NOT bring down the entire logical connection between node A and node B → data can still continue to traverse over the still-connected connection link!
- 2.) If there is heavy network traffic (aka overhead) is present:
 - a. Too many packets travelling between node A and node N? It is possible to divide the bulk traffic between the two or more unique routes and not cause the same network congestion without the packet traffic path division between node A, node B, . . . node N-1 and thus node N. Spanning Tree Protocol and the concept of “load balancing” in the routing protocol you chose (ex: RIP, OSPF, IGRP, EIGRP, etc.).

<Source: <https://www.cisco.com/c/en/us/support/docs/ip/border-gateway-protocol-bgp/5212-46.html>>

Question assigned to the following page: [3](#)

Problem Set #5

Ch 7/#16:

(Using Dijkstra's Algorithm...)

a.) Total # of simple paths from node A to node G?

7 paths exist:

1.) ABCG

2.) ABECG

3.) ABEDCG

4.) AFG

5.) ADCG

6.) ADEBCG

7.) ADECG

b.) Total # of shortest paths from node A to node G?

Two paths exist:

1.) ADECG (cost = 10)

2.) ABECG (cost = 10)

c.) If node E should fail, then both ADECG and ABECG shortest paths will also fail, plus other existing paths.

Valid working paths remaining?

1.) ABCG (cost = 11)

2.) ADCG (cost = 11)

3.) AFG (cost = 11)

Question assigned to the following page: [3](#)

Problem Set #5

Given

Simple Path 5 (v_2, t_1):
A D E B C G (cost = 15)

Simple Path 6 (v_2, t_2):
A D E C G (cost = 10)
is an SPL

Simple Path 7 (v_1, t_1):
A B E D C G (cost = 13)

Q # of simple paths from node A to node G that do NOT repeat a node?

Node	Iteration 0	Iteration 1	Iteration 2	Iteration 3	Iteration 4	Iteration 5
A	0	X	X	X	X	X
B	∞	5		5 (v_2, t_1)	X	X
C	∞	∞	9 (v_1, t_1)	8 (v_1, t_2)	11 (v_1, t_1)	X
D	∞	6		8 (v_1, t_1)	11 (v_1, t_2)	X
E	∞	∞	7 (v_1, t_1)	10 (v_2, t_2)	10 (v_2, t_1)	X
F	∞	3	X	X	X	X
G	∞	∞	11	11 (v_2, t_1)	10 (v_1, t_2)	13 (v_1, t_1)

Simple Path 1: A F G (cost = 11)

Simple Path 2 (v_1, t_2): A B E C G (cost = 10) is an SPL

Simple Path 3 (v_2, t_1): A B C G (cost = 11)

Simple Path 4 (v_1, t_1): A D C G (cost = 11)

Looped Path 1 (v_1, t_2):
A B E C D — Dead End!

Looped Path 2 (v_2, t_2):
A B C E D — Dead End!

Looped Path 3 (v_1, t_1): A B C D E

Question assigned to the following page: [3](#)

Problem Set #5

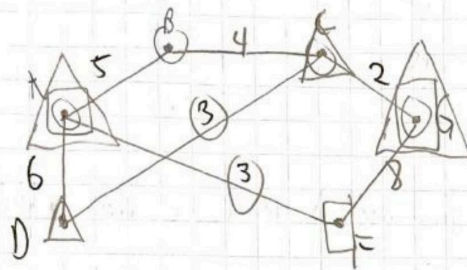
(b) # of shortest paths (SP's) from node A to node G?

Two! (1) ABECG (cost = 10)

(2) ADECG (cost = 10)

(c) If node E itself should fail, what remaining paths will still route packets from node A to node G?

New topology?



Valid Paths remaining from node A to node G?

Three! (1) A(B)C(G) (cost = 11) <path 1>

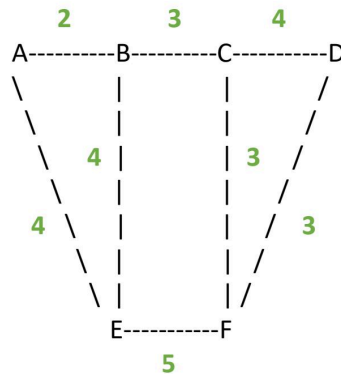
(2) A(F)G (cost = 11) <path 2>

(3) A(D)G (cost = 11) <path 3>

Question assigned to the following page: [4](#)

Problem Set #5

Ch. 7/#17:



Overall, there are cases when this routing approximation algorithm is not too far off from the true shortest path from a given source node to a given destination node.

Using the simple heuristic from node A to node D:

1.) At A, choose B (cost = 2)

2.) At B, choose C (cost = 3)

3.) At C, choose F (cost = 3)

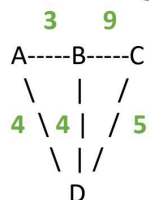
4.) At F, choose D (cost = 3)

*Total Cost from node A to node D? (cost: 2 + 3 + 3 + 3 = **11 total units**)

Using Dijkstra's shortest path first algorithm:

*D→C→B→A yields shortest path ABCD (cost: 4 + 3 + 2 = **9 total units**)

But for a slightly different topology and a new given path from A→C, looping errors will arise:



1.) At A, choose B (cost = 3)

2.) At B, choose D (cost = 4)

3.) At D, choose A (cost = 4) **Loop detected! Heuristic fails here!!!**

Question assigned to the following page: [5](#)

Problem Set #5

Ch. 7/#20:

The advantages of packet encapsulation of messages are:

- 1.) The ability to send each message piece through a different route to when using a load balancer (software or hardware based) in your chosen routing algorithm or physical network topology.
- 2.) Given an error, only the packet containing that error is dropped. The message, as a whole, will be transmitted successfully (due to TCP/IP flow control, error correction properties).
- 3.) Out-of-order transmission of packets from a given program 1, program 2, , program n—is now possible and can be tied directly to Quality of Service weighted values and network queuing methods (ex: Class-Based Weighted Fair Queueing in Cisco IOS).

Now, packets are assigned an appropriate weighted numeric value, according to their type: data, voice, or video.

This ensures that voice message transmission is tolerant of variable ping latency and jitter values, while still maintaining reasonable audio quality.

Similar connection attributes also ensure video message transmission is tolerant of variable ping latency and jitter values, while still maintaining reasonable audiovisual quality.

Data, usually assigned the lowest packet priority, will still get through in its entirety but will not tie up the total network bandwidth with just a single large file transfer (say no to BitTorrent, please!).

https://www.cisco.com/en/US/docs/ios/12_0t/12_0t5/feature/guide/cbwfq.html#wp17641

Question assigned to the following page: [6](#)

Problem Set #5

Ch. 7/#21: ARPANet Pioneer: Vinton Cerf

Vinton Cerf, co-creator of the TCP/IP stack protocol, is responsible for the very foundation-level transmission and network protocol upon which surfing the web, checking our email, or streaming a Netflix movie are **all** made possible. But he did not come out of nowhere to create this masterwork. Even in high school, he worked on various projects tied to NASA and the Department of Defense.

His focus continued in an undergraduate mathematics program at Stanford University. Later, he would matriculate to UCLA, where he would help physically connect the first two network nodes on the original ARPANet and code the transmission protocol needed for node-to-node message transmission.

Yet TCP/IP was born out of joint collaboration. First, just between Cerf and Bob Kahn, the operating systems guru of various computing architectures of the time. Cerf already had the precursor to TCP, NCP from his work on ARPANet in 1972. But Kahn had the idea of creating a single communications protocol that would run on any hardware and be platform-independent, such that a computer of brand X with a said operating system A could freely talk to a computer of brand Y with said operating system B. This formed the basic principle of TCP/IP: a common network protocol suite that could work on any machine running any software platform.

Previous to the creation of the DoD TCP/IP standard in 1974, this was not feasibly possible: unique code would be customized to manage and internally translate each unique end running a so called “end-to-end” protocol, per connection link. As you can see, writing custom communication protocols and network device drivers for every unique node on a medium to large network can be cumbersome and extremely time-consuming. By creating and formalizing one common standard for all internetwork traffic on ARPANet and beyond.

Cerf and Kahn, in the 1970s, could not yet imagine a world where LANs, MANs, & WANs would become commonplace in nearly all aspects of modern life. Heck, Ethernet itself (IEEE 802.3), needed to establish the OSI model’s Layer 2: Data Link Layer standard, was invented nearly at the same time of TCP/IP, in 1973, by Xerox PARC’s Robert Metcalfe. Only over time and via collaboration with other scientists/engineers could the various standards that would eventually form the basis of the modern-day Internet be created. Nobody could do it alone. Additionally, the development from ARPANet→Internet was nowhere near linear each year of its existence. From 1972-1976, Cerf began to teach and lead his own research group at Stanford University. To Vinton Cerf, TCP/IP was just a means to an end: a protocol necessitated by a DARPA research grant that he was commissioned to complete: no more, no less.